SIMULATED ANNEALING 8-QUEENS

```
import random
import math


def cost(state):
    # number of pairs of queens attacking each other
    h = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == j - i:
                h += 1
    return h


def print_board(state):
    n = len(state)
    for r in range(n):
        print(" ".join("Q" if state[c] == r else "." for c in range(n)))
    print()


def simulated_annealing(state, T=5.0, cooling=0.999, T_min=0.001, show=5):
    current = state[:]
    steps = 0
    while T > T_min:
        h = cost(current)
        if h == 0:
            return current, steps                # goal reached
```

```
    # random neighbor: move one random queen to a random row
    nb = current[:]
    col = random.randrange(len(nb))
    nb[col] = random.randrange(len(nb))

    old = current[:]
    delta = cost(nb) - h
    if delta < 0:
        accept, p = True, 1.0
    else:
        p = math.exp(-delta / T)
        accept = random.random() < p
    if accept:
        current = nb

    if steps < show:                    # show only the first few steps
        print(f"Step {steps+1}: T={T:.3f}")
        print(f"   current  = {old}  cost = {h}")
        print(f"   neighbor = {nb}  cost = {h + delta}")
        print(f"   delta = {delta:+d}, accept prob = {p:.3f} -> {'ACCEPTED' if accept else 'REJECTED'}")
    elif steps == show:
        print("... (remaining steps not shown) ...\n")

    T = T * cooling
    steps += 1
return current, steps
```

```
n = 8
initial = [random.randrange(n) for _ in range(n)]

print("INITIAL STATE:", initial, "| Cost:", cost(initial))
print_board(initial)

result, steps = simulated_annealing(initial)

print("GOAL STATE: Cost = 0 (no queens attacking each other)\n")
print("FINAL STATE:", result, "| Cost:", cost(result), "| Steps:", steps)
print_board(result)
```

OUTPUT :

```
INITIAL STATE: [0, 4, 6, 0, 7, 2, 7, 2] | Cost: 5
Q . . Q . . . .
. . . . . . . .
. . . . . Q . Q
. . . . . . . .
. Q . . . . . .
. . . . . . . .
. . Q . . . . .
. . . . Q . Q .

Step 1: T=5.000
   current  = [0, 4, 6, 0, 7, 2, 7, 2]  cost = 5
   neighbor = [0, 4, 6, 6, 7, 2, 7, 2]  cost = 7
   delta = +2, accept prob = 0.670 -> REJECTED
Step 2: T=4.995
   current  = [0, 4, 6, 0, 7, 2, 7, 2]  cost = 5
   neighbor = [0, 4, 6, 0, 7, 1, 7, 2]  cost = 3
   delta = -2, accept prob = 1.000 -> ACCEPTED
Step 3: T=4.990
   current  = [0, 4, 6, 0, 7, 1, 7, 2]  cost = 3
   neighbor = [0, 4, 6, 6, 7, 1, 7, 2]  cost = 6
   delta = +3, accept prob = 0.548 -> ACCEPTED
Step 4: T=4.985
   current  = [0, 4, 6, 6, 7, 1, 7, 2]  cost = 6
   neighbor = [0, 4, 6, 6, 7, 1, 2, 2]  cost = 8
   delta = +2, accept prob = 0.670 -> ACCEPTED
Step 5: T=4.980
   current  = [0, 4, 6, 6, 7, 1, 2, 2]  cost = 8
   neighbor = [0, 4, 6, 6, 7, 1, 2, 7]  cost = 8
   delta = +0, accept prob = 1.000 -> ACCEPTED
... (remaining steps not shown) ...

GOAL STATE: Cost = 0 (no queens attacking each other)

FINAL STATE: [5, 2, 0, 7, 4, 1, 3, 6] | Cost: 0 | Steps: 3056
. . Q . . . . .
. . . . . Q . .
. Q . . . . . .
. . . . . . Q .
. . . . Q . . .
Q . . . . . . .
. . . . . . . Q
. . . Q . . . .
```